
ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к проекту по предмету «Информатика»
профиль «Информационная безопасность»
по теме:

**Программное средство автоматизированного аудита и
харденинга операционной системы на основе открытых
стандартов SCAP**

Направление: Blue Team

Реферат

Отчёт 46 с., 6 рис., 4 табл., 12 источн., 2 прил.

SCAP, OVAL, XCCDF, АУДИТ БЕЗОПАСНОСТИ, ХАРДЕНИНГ, УЯЗВИМОСТИ, МИСКОНФИГУРАЦИИ, ПАТЧИНГ, REMEDIATION, openSCAP

Объектом исследования является процесс автоматизированного аудита и харденинга операционной системы на основе стандартизированного SCAP-контента.

Цель работы - разработка программного средства автоматизированной проверки операционной системы на наличие уязвимостей и мисконфигураций на основе стандартов SCAP с возможностью формирования отчёта и автоматического устранения выявленных несоответствий.

В процессе работы был изучен открытый стандарт SCAP, в частности OVAL и XCCDF, а также проведён анализ существующих решений в области автоматизации аудита безопасности.

В результате выполнения проекта было разработано программное средство, позволяющее автоматически загружать SCAP-контент для выбранной операционной системы, выполнять проверки по OVAL-дефинициям и XCCDF-правилам, автоматически адаптировать контент к целевой платформе при помощи патчинга SCAP-контента, применять remediation при наличии соответствующего контента и выводить результаты в удобочитаемом табличном виде.

Основными показателями продукта являются поддержка открытых стандартов SCAP, возможность автоматического патчинга, поддержка remediation, а также возможность расширения функциональности за счёт добавления новых платформ, источников SCAP-контента и модульности архитектуры.

Разрабатываемая программа является превентивной мерой, поскольку за счёт автоматизированного аудита и харденинга устраняет уязвимости и мисконфигурации, чем снижает поверхность атаки и минимизирует вероятность эксплуатации типовых техник злоумышленника. В работе показано, что программа защищает от нескольких техник по MITRE ATT&CK, связанных с эксплуатацией публично доступных сервисов, повышением привилегий, изменением настроек фаерволла, а также небезопасной конфигурацией sudo.

Оглавление

ИСПОЛЬЗУЕМЫЕ ТЕРМИНЫ И СОКРАЩЕНИЯ.....	4
ВВЕДЕНИЕ.....	6
1. ИССЛЕДОВАНИЕ ПРОБЛЕМЫ.....	7
1.1 Проблема и её актуальность	7
1.2 ПО для автоматического анализа уязвимостей	7
1.3 Протокол SCAP.....	8
1.4 Анализ аналогичных решений.....	11
1.5 Аудитория проекта.....	12
2. РЕШЕНИЕ ПРОБЛЕМЫ.....	14
2.1 Формализация проблемы	14
2.2 Описание алгоритма решения проблемы.....	15
2.3 Описание существующих алгоритмов.....	16
2.4 Модель угрозы, MITRE ATT&CK	17
3. ОПИСАНИЕ РЕАЛИЗАЦИИ.....	20
3.1 Техническое задание.....	20
3.2 Принцип работы программы	24
3.3 Доказательство работоспособности.....	25
4. Тестирование.....	27
4.1 Цели тестирования.....	27
4.2 Функциональное тестирование	27
4.3 Пользовательское тестирование	30
4.4 Анализ результатов тестирования	30
4.5 Перспективы развития проекта.....	31
ЗАКЛЮЧЕНИЕ.....	32
СПИСОК ЛИТЕРАТУРЫ	34
ПРИЛОЖЕНИЕ А.....	36
ПРИЛОЖЕНИЕ Б	43

ИСПОЛЬЗУЕМЫЕ ТЕРМИНЫ И СОКРАЩЕНИЯ

Агрегация результатов - сбор данных из файлов результатов.

Аудит (информационной безопасности) - регулярные проверки, отчётность, анализ состояния защищённости.

Вендор - поставщик продукта/решения.

Киберугрозы - угрозы атак на ИС.

Мисконфигурация - небезопасная/ошибочная настройка.

ОС - операционная система.

Оценка соответствия конфигурации - проверка системы на соответствие политике/профилю требований.

ПО - программное обеспечение.

Патчинг - процесс внесения изменений в программное обеспечение. В контексте проекта это изменение SCAP-контента для его корректной работы на целевой системе

Проприетарный - закрытый (непрозрачный) механизм/формат/подход вендора.

Сигнатурное/базовое сопоставление - подход: сбор версий ПО и сравнение с базой уязвимых версий.

Стандартизация проверок/данных - унификация форматов правил и результатов.

Угроза - мнимая или реальная опасность.

Уязвимость - это слабость в системе, которая может привести к реализации угрозы.

Харденинг - укрепление конфигурации ОС/системы (снижение поверхности атаки).

C++ - язык программирования.

CCE - идентификаторы параметров/настроек конфигурации.

CIS - набор бенчмарков/рекомендаций по безопасной конфигурации.

CPE - стандартизированный формат идентификации платформ, операционных систем и программного обеспечения

CVE - идентификаторы известных уязвимостей.

CVSS - методика оценки критичности уязвимостей.

CWE - классификация типов слабостей (уязвимых паттернов).

Dart - язык программирования.

DataStream - формат представления SCAP-контента, объединяющий в себе сразу несколько составляющих.

Electron / Flutter - фреймворки для разработки кроссплатформенных приложений.

Firewall - межсетевой экран, средство для фильтрации сетевого трафика.

GPL - лицензия GNU General Public License.

GTK - фреймворк GUI.

HTML - формат данных.

JS - JavaScript.

JSON - формат данных.

LGPL - лицензия GNU Lesser General Public License.

Linux / Windows / macOS - платформы.

MIT / BSD - типы лицензий свободного программного обеспечения.

MITRE ATT&CK - глобальная база знаний тактик и техник злоумышленников.

Open Source - открытый исходный код.

openSCAP - инструментарий/движок для обработки SCAP-контента.

OVAL - Open Vulnerability and Assessment Language: язык (стандарт) описания и выполнения технических проверок состояния системы.

OVAL-дефиниция - отдельное правило/описание проверки в формате OVAL.

PCI DSS - Payment Card Industry Data Security Standard: стандарт безопасности данных индустрии платёжных карт.

PyQt - Python-библиотека (биндинги) для Qt.

PySide - Python-библиотека (биндинги) для Qt.

Qt - фреймворк для графического интерфейса.

RedCheck, XSpider, ScanOVAL, MaxPatrol 8, MaxPatrol VM - названия продуктов/аналогов.

RHEL (Red Hat Enterprise Linux) / Debian / Rocky Linux - дистрибутивы операционной системы Linux.

SCAP - Security Content Automation Protocol: набор открытых стандартов для представления/обмена данными ИБ и автоматизации проверок.

SCAP-интерпретатор / движок - компонент, который исполняет проверки по SCAP-контенту.

SCAP-контент - набор машиночитаемых правил/описаний проверок (OVAL/XCCDF).

Remediation - действия по устранению несоответствий (в т.ч. автоматическое исправление).

Vulnerability management - управление уязвимостями.

XCCDF - Extensible Configuration Checklist Description Format: формат описания чек-листов/политик проверок, профилей и результатов; поддерживает remediation.

XCCDF-профиль - набор (выборка) правил XCCDF для конкретной политики.

XCCDF-правило - отдельное требование/проверка в XCCDF.

ВВЕДЕНИЕ

В наше время информационные технологии используются во всех областях, начиная от бизнеса и заканчивая государственными организациями. Цифровая структура всех этих областей растёт крайне стремительно, что также повышает требования к обеспечению защищённости систем. При этом киберугрозы развиваются также быстро, как и технологии: злоумышленники используют широкий спектр методов - от известных уязвимостей до ошибок конфигураций. В таких условиях требуется непрерывный мониторинг информационной безопасности систем, обычно это достигается за счёт регулярных проверок, формирования отчётов и последующего устранения выявленных уязвимостей.

Важной составляющей таких процессов является стандартизация данных и проверок. Использование стандартов позволяет унифицировать проверки и их результаты, а также снизить зависимость от конкретных инструментов. В области автоматизации контроля безопасности существует единственный набор открытых стандартов SCAP, предназначенный для описания и обмена данными по безопасности, а также для автоматизирования проверок.

Цель работы: Создание программного средства автоматизированного аудита и харденинга операционной системы на основе стандартов SCAP

Задачи:

1. Изучить проблему и её актуальность
2. Изучить протокол SCAP
3. Реализовать модуль проверки по OVAL/XCCDF-контенту
4. Реализовать модуль для автоматического патчинга
5. Реализовать модуль агрегации результатов проверки из XML файлов
6. Реализовать визуальный интерфейс
7. Провести функциональное и пользовательское тестирование

1. ИССЛЕДОВАНИЕ ПРОБЛЕМЫ

1.1 Проблема и её актуальность

С каждым днём цифровая инфраструктура компаний растёт, число угроз увеличивается, а злоумышленники активно используют публично известные уязвимости и ошибки конфигураций. На этом фоне заметна общая тенденция - темпы появления уязвимостей увеличиваются. По графику с Рис. П1.1 видно, что лишь за период с 2015 по 2019 год количество опубликованных уязвимостей выросло на 200% (с 6000 до 18000), а с 2019 по 2023 - ещё на 55% (с 18000 до 28000). Это означает, что объём информации, которую необходимо анализировать и обрабатывать специалистам из области информационной безопасности, растёт непрерывно уже более 10 лет, что делает задачу автоматизации процессов выявления и устранения уязвимостей и мисконфигураций особенно актуальной.

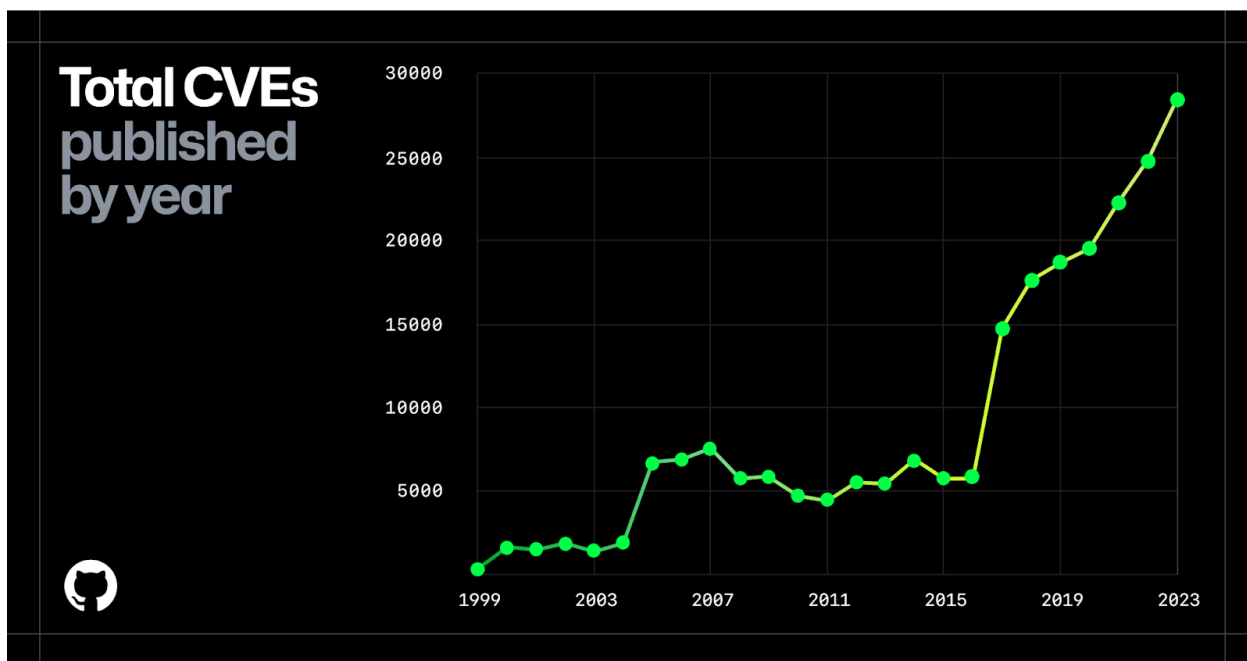


Рисунок 1.1 - График количества найденных уязвимостей по годам

1.2 ПО для автоматического анализа уязвимостей

На данный момент существует крайне большое количество инструментов для поиска и управления уязвимостями в информационных системах. Среди распространённых решений можно выделить Redcheck, MaxPatrol 8, XSpider, MaxPatrol VM. Эти продукты помогают проводить аудит уязвимостей и формировать отчётность, однако у большинства из них есть ограничения, которые становятся критичными при построении процессов информационной безопасности.

Основные недостатки подобных решений:

- Проприетарные методы анализа. Внутренние механизмы проверки и логика обнаружения зачастую закрыты. Это затрудняет независимую верификацию результатов, а также ограничивает расширение или изменение функционала программы без вмешательства вендора.

- Собственные базы данных и форматы представления контента для проверки. Многие вендоры агрегируют сведения об уязвимостях, например CVE, в собственные базы/JSON/XML-инструкции. Данный подход приводит к зависимости от конкретного вендора и затруднению разработки инструмента, работающего с программой или её контентом.

Именно для избегания данных проблем разработанная программа на каждом этапе будет опираться на открытые стандарты SCAP.

1.3 Протокол SCAP

SCAP (Security Content Automation Protocol, протокол автоматизации управления данными безопасности) - это набор открытых стандартов, определяющих технические спецификации для представления, описания и обмена данными в области безопасности. Его задача - обеспечить единый подход к автоматизации проверок: от описания уязвимостей и конфигураций до форматов тестов и результатов оценки. В состав SCAP входят различные компоненты, среди которых:

- CVE - идентификаторы известных уязвимостей;
- CWE - классификация типов слабостей;
- CCE - идентификаторы параметров конфигураций;
- CVSS - методика оценки критичности;
- XCCDF - формат описания контрольных листов и политик проверок;
- OVAL - язык описания и выполнения технических проверок состояния системы.

При создании программы ключевыми компонентами будут являться CVE, OVAL и XCCDF, потому что именно благодаря им программа будет проверять систему на наличие уязвимостей и мiskonfigураций по машиночитаемому описанию проверок. Роль этих трёх

компонентов стоит разобрать как в рамках разбора работы протокола SCAP, так и в рамках реализации программы:

- CVE предоставляет унифицированные идентификаторы уязвимостей и базовую справочную информацию. Большинство продуктов на рынке ограничиваются только агрегацией информации из CVE и последующим формированием собственных правил. Но в протоколе SCAP ролью CVE является только базовое описание уязвимости и предоставление ей идентификатора

- OVAL (Open Vulnerability and Assessment Language) - это открытый стандартизированный язык, который в машиночитаемом виде предоставляет информацию как проверять систему на наличие уязвимостей. OVAL имеет одинаковую структуру вне зависимости от вендора.

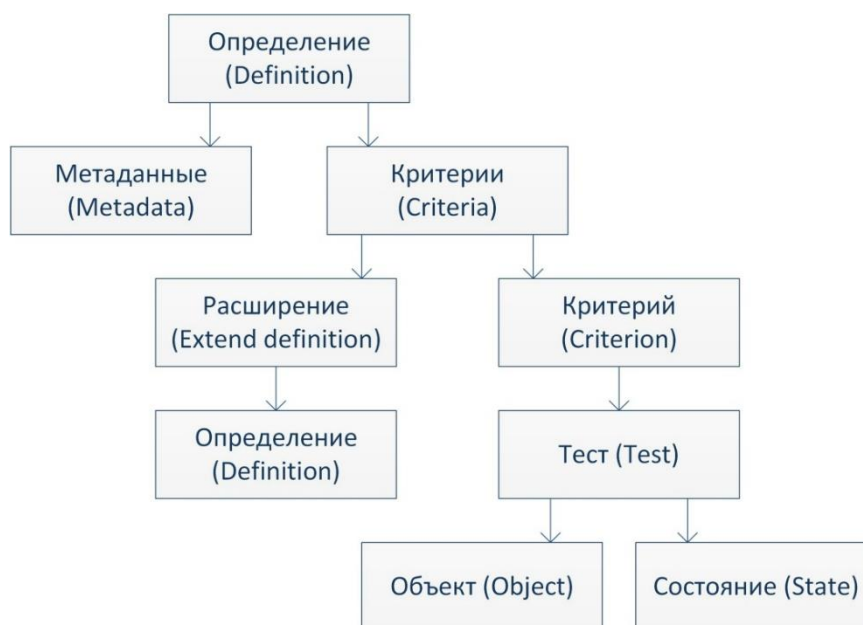


Рисунок 1.2 - Структура OVAL

- XCCDF (Extensible Configuration Checklist Description Format) - это формат, который позволяет описывать политики и структуру проверок: какие требования проверять, как их группировать, какие профили использовать, как формировать результат оценки. Также благодаря XCCDF можно проводить remediation (устранение уязвимостей).



Рисунок 1.3 - Структура XCCDF

Для разработки программы по автоматизации обнаружения уязвимостей необходимо изучить общий процесс работы SCAP (Рис. П2.1). Также в рамках целей проекта необходимо разобрать техническую реализацию взаимодействия OVAL-дефиниций и XCCDF-контента (Рис. П2.2).

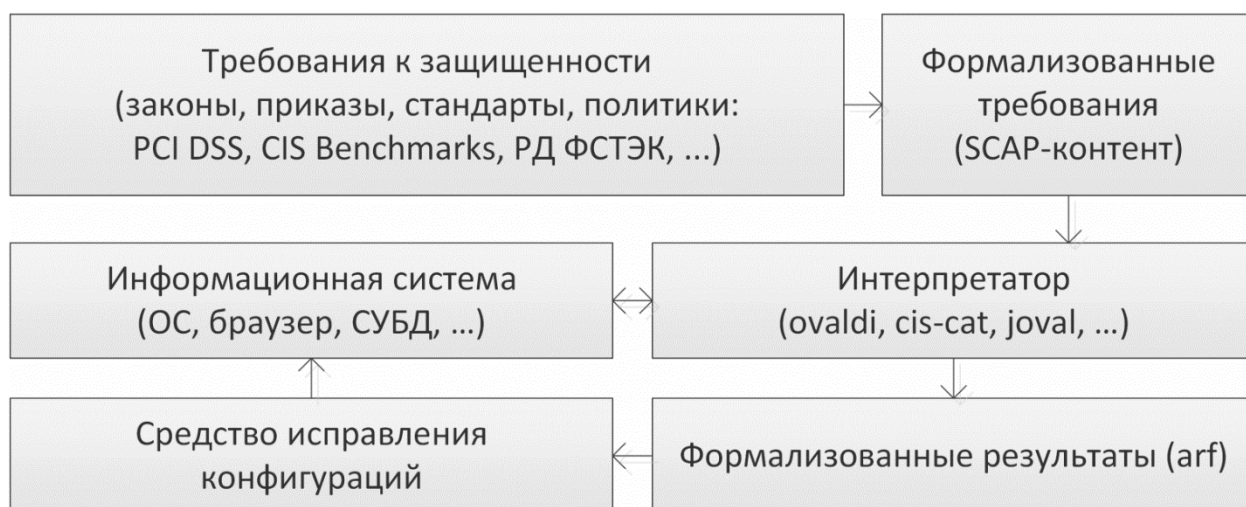


Рисунок 1.4 - Процесс работы протокола SCAP

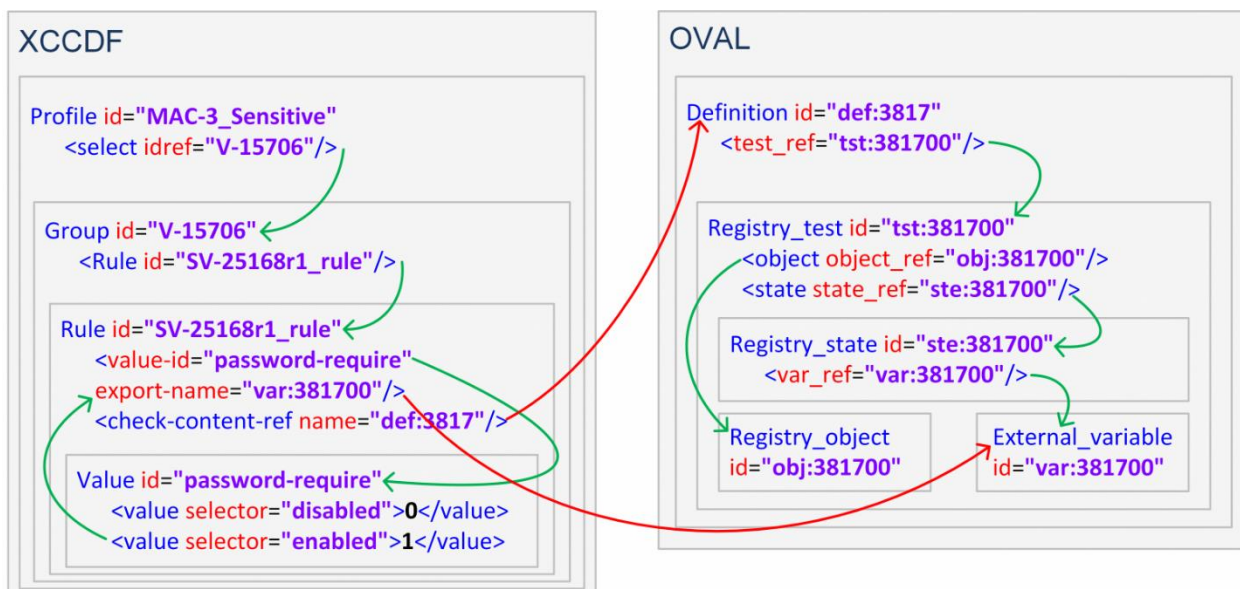


Рисунок 1.5 - Структура взаимодействия XCCDF и OVAL на примере CIS Benchmark

1.4 Анализ аналогичных решений

Для определения новизны и уникальности создаваемой программы был проведён анализ существующих решений для поиска и управления уязвимостями. В качестве аналогов были рассмотрены: RedCheck, XSpider, ScanOVAL, MaxPatrol VM.

Таблица 1 - Сравнение с аналогами

Инструмент	Поддержка SCAP	OpenSource	Цена	Remediation	Tailoring
Xspider	Нет	Нет	Платно	Нет	Нет
RedCheck	Да	Нет	Платно	Да	Нет
ScanOVAL	Частичная (только OVAL-контент от ФСТЭК)	Нет	Бесплатно	Нет	Нет
MaxPatrol VM	Нет	Нет	Платно	Да	Нет
Проектная программа	Да	Да	Бесплатно	Да	Да

Сравнение выполнено по следующим критериям: поддержка SCAP, открытость исходного кода (Open Source), стоимость, наличие remediation (механизмов устранения или рекомендаций по устранению), а также патчинга (возможности адаптации и кастомизации проверочного контента).

XSpider является примером классического сканера уязвимостей. Инструмент ориентирован на собственные механизмы обнаружения и не использует SCAP-контент, поэтому в рамках данного подхода в нём отсутствуют поддержка патчинга и remediation. Решение не является Open Source и распространяется на коммерческой основе.

RedCheck относится к решениям, поддерживающим SCAP-контент. Продукт предоставляет remediation функционал, однако не реализует механизмы патчинга для адаптации проверочного контента под целевую систему. Кроме того, решение является закрытым и коммерческим.

ScanOVAL это бесплатный инструмент, ориентированный на проверку системы по OVAL-дефинициям, что означает только частичную поддержку SCAP. Поскольку программа не поддерживает XCCDF-контент, в ней отсутствуют возможности патчинга и remediation в контексте XCCDF-проверок.

MaxPatrol VM представляет собой коммерческую платформу управления уязвимостями, ориентированную на полный цикл vulnerability management. За счёт этого в продукте реализованы механизмы remediation. Вместе с тем, MaxPatrol VM не предполагает использования SCAP-контента.

Таким образом, рассмотренные решения либо совсем не поддерживают SCAP-контент, либо являются закрытыми коммерческими продуктами. Кроме того, большинство программ, поддерживающих SCAP-контент, не обладают возможностью автоматической адаптации (патчинга) SCAP-контента под конкретную операционную систему.

Следовательно, из совокупности данных параметров формируется новизна проектной программы сразу с 2 сторон – как со стороны продукта, так и со стороны технического подхода. Со стороны продукта новизна заключается в сочетании поддержки SCAP-контента и распространения программы в формате Open Source. Со стороны технической реализации новизна обусловлена механизмом автоматического патчинга, позволяющим существенно расширить круг платформ, для которых может быть использован SCAP-контент.

1.5 Аудитория проекта

Поскольку разрабатываемая программа предназначена для автоматизированного аудита системы и поддержки харденинга на основе SCAP-контента (OVAL/XCCDF), её

целевая аудитория напрямую связана с задачами контроля защищённости, соответствия требованиям и регулярной проверки конфигураций и уязвимостей. Соответственно, основными группами пользователей являются:

- Аудиторы информационной безопасности. Для них актуальна задача получения воспроизводимых результатов проверок, формирования отчётности и подтверждения выполнения требований (политик безопасности, внутренних регламентов и внешних стандартов). Использование стандартизированного контента облегчает сравнение результатов между системами и аудитами.

- Системные администраторы. Для них программа полезна как средство быстрой проверки состояния системы, выявления уязвимостей и мисконфигураций, а также устранения уязвимостей, что упрощает поддержание безопасности информационных систем.

Аудитория проекта является профессиональной и относительно ограниченной, поскольку ориентирована на специализированные задачи, однако программа имеет широкое применение в организациях, где нужен постоянный аудит и стандартизированная проверка безопасности.

2. РЕШЕНИЕ ПРОБЛЕМЫ

2.1 Формализация проблемы

В рамках данной программы решается задача автоматизированной проверки системы на наличие уязвимостей и мисконфигураций на основе стандартизированного SCAP-контента. Для исключения неоднозначности и возможности воспроизведения алгоритма задача формализуется в виде входных данных, выходных данных, ограничений и алгоритма обработки:

Входные данные:

1. Целевая система S - проверяемая операционная система
2. SCAP-контент C, включает набор OVAL-дефиниций $D = \{d1, d2, d3..., dn\}$ и XCCDF-правил $R = \{r1, r2, r3...rn\}$, в случае XCCDF также содержат данные remediation $RD = \{rd_r1, rd_r2, rd_r3...rd_rn\}$
3. Параметры запуска P: ОС, профиль XCCDF, режим выполнения проверок.

Выходные Данные:

- Результаты при проведении проверки по OVAL: для каждой дефиниции di вычисляется статус $Result_OVAL \in \{true, false, unknown, error, notapplicable\}$, где true - уязвимость/несоответствие присутствует, false - уязвимость/несоответствие отсутствует, unknown/error/notapplicable - невозможность или неприменимость проверки.

- Результаты при проведении проверки по XCCDF: для каждого правила ri вычисляется статус $Result_XCCDF \in \{pass, fail, notselected, notapplicable, error\}$, где pass - правило успешно выполнилось, fail - правило выполнилось не успешно, notselected, notapplicable, error - невозможность или неприменимость проверки, notselected – правило не было выбрано в XCCDF профиле. При включённом remediation для каждого fail ri выполняется rdi

3. Отчёт Report в формате таблицы, содержащей для каждой проверки:

- При проверке OVAL-дефиниций Report содержит: cve_id, di, описание уязвимости

- При проверке XCCDF-правил Report содержит: XCCDF_rule, ri, описание правила, уровень важности правила.

2.2 Описание алгоритма решения проблемы

Формализованный общий алгоритм работы программы:

1. Выбор контента и подготовка среды: определяется платформа S, загружается SCAP-контент C, задаются параметры запуска P
2. Патчинг (при необходимости): если контент содержит идентификаторы платформы, не совпадающие с целевой системой, контент C изменяется для успешной работы SCAP-контента C на S.
3. Выполнение OVAL-проверок: для каждой дефиниции di вычисляются требуемые свойства системы (версии, параметры, состояния объектов), после чего определяется Result_OVAL(di)
4. Выполнение XCCDF-проверок: на основе выбранного профиля вычисляется результат для каждого правила ri, после чего определяется Result_XCCDF(ri). Если включен remediation и Result_XCCDF(ri) для отдельного правила вернул fail – выполняется rdi при наличии remediation контента в XCCDF.
5. Формирование отчёта: результаты Result_OVAL и Result_XCCDF агрегируются и преобразуются в удобный для анализа вид (таблица).
6. Вывод пользователю: отчёт предоставляется пользователю в человекочитаемом виде внутри интерфейса программы.

Описание алгоритма решения проблемы:

Режим 1: Проверка по запросу (OVAL)

Данный режим используется для проверки системы на наличие CVE и выполнения других проверок и запускается пользователем вручную.

1. Выбор ОС и контента. Пользователь выбирает целевую платформу (ОС/дистрибутив), далее программа скачивает SCAP-контент.

2. Выполнение проверок. Программа последовательно выполняет OVAL-проверки (получая факты о системе: версии ПО, параметры, состояния объектов) и записывает их в список с результатами проверки
3. Формирование результатов. Пользователь получает результат в виде таблицы: идентификаторы CVE, статусы (true/false и т.д.), описания CVE.

Режим 2: Проверка по запросу (XCCDF + remediation)

Данный режим используется для проверки системы на соответствие XCCDF-профилю и запускается пользователем вручную.

1. Выбор ОС и контента. Пользователь выбирает целевую платформу (ОС/дистрибутив), далее программа скачивает SCAP-контент.
2. Адаптация (патчинг). Если выбранный XCCDF-контент предназначен для близкой платформы, но не применяется из-за различающегося идентификатора системы, программа выполняет автоматическую адаптацию контента (патчинг), обеспечивая возможность корректного запуска проверок на текущей системе.
3. Выполнение проверок. Программа последовательно выполняет OVAL-проверки (получая факты о системе: версии ПО, параметры, состояния объектов), затем вычисляет результаты XCCDF-правил на основе выбранного профиля и записывает их в список с результатами проверки.
4. Формирование результатов. Пользователь получает результат в виде таблицы: идентификаторы проверок, описания, статусы (pass/fail и т.д.), важность проверок.

2.3 Описание существующих алгоритмов

Помимо алгоритмов, взятых из стандартов SCAP, таких как OVAL evaluation и XCCDF evaluation также стоит рассмотреть несколько других методов анализа системы на небезопасные конфигурации и уязвимости:

Сигнатурное/базовое сопоставление - этот алгоритм заключается в сборе информации о ПО в системе (инвентаризация пакетов, версий) и сопоставления их с базой уязвимых версий программ. Данный алгоритм используется только для нахождения CVE, позволяет быстро масштабировать базу данных, но обладает минусами в виде большого

числа ложных срабатываний, сложностью написания сложных условий и плохой возможностью описания конфигурационных требований к системе.

Алгоритмы оценки соответствия конфигурации – эти алгоритмы заключаются в наличии политики/профиля требований, для каждого требования запускается проверка состояния, затем агрегируются результаты (pass/fail/not applicable) и строится отчёт по профилю.

SCAP алгоритмы же, как мы можем видеть, объединяют самые лучшие практики из множества уже существующих алгоритмов нахождения уязвимостей и небезопасных конфигураций

2.4 Модель угрозы, MITRE ATT&CK

Проект предназначен для повышения защищённости Linux-систем за счёт регулярного автоматизированного аудита и харденинга на основе стандартизированного SCAP-контента. Для формального описания того, какие этапы атаки усложняет харденинг, используется структура фреймворка MITRE ATT&CK.

Целевой класс угроз: Угрозы, использующие публично известные уязвимости и мисконфигурации.

Общая цель злоумышленника: использовать известные уязвимости и/или ошибки конфигурации для получения доступа к системе, расширения прав, закрепления и дальнейшего выполнения действий в инфраструктуре.

Портрет злоумышленника: наиболее вероятным нарушителем является атакующий с базовым или средним уровнем квалификации, использующий общедоступные эксплойты и инструменты для эксплуатации публично известных уязвимостей и мисконфигураций.

1. Соответствие тактикам и техникам MITRE ATT&CK

Ниже приведены примеры техник MITRE ATT&CK, для которых результаты аудита и remediation по SCAP-контенту выступают превентивной контрмерой (через харденинг и устранение условий атаки):

1. T1190: Exploit Public-Facing Application

Что соответствует: злоумышленники пытаются эксплуатировать слабости в публично доступных сервисах, включая уязвимости ПО и ошибки конфигурации.

Как противодействует проект: проверяет состояние системы и конфигураций по OVAL/XCCDF (версии, параметры, настройки сервисов) и применяет remediation, уменьшая вероятность наличия уязвимых/небезопасно настроенных компонентов на хосте.

2. T1068: Exploitation for Privilege Escalation

Что соответствует: эксплуатация уязвимостей ПО/ОС для повышения привилегий.

Как противодействует проект: за счёт проверок уязвимых состояний и их последующего исправления снижает вероятность наличия известных уязвимых конфигураций, используемых для эскалации.

3. T1562.004 — Impair Defenses: Disable or Modify System Firewall

Что соответствует: злоумышленник отключает или изменяет правила системного файрвола, чтобы обойти ограничения сетевого трафика.

Как противодействует проект: через SCAP-контент выполняет аудит состояния файрвола и правил (включён ли сервис, разрешённые сервисы/порты и т.д.) и применяет remediation, возвращая конфигурации к заданным стандартам.

4. T1548.003 — Abuse Elevation Control Mechanism: Sudo and Sudo Caching

Что соответствует: злоумышленник использует особенности sudo и настройки sudoers, чтобы выполнить команды от имени другого пользователя или запустить процессы с более высокими привилегиями.

Как противодействует проект: через SCAP-контент проверяет и исправляет конфигурацию sudo.

2. Вывод по модели угроз

Разрабатываемая программа является превентивной мерой: за счёт автоматизированного аудита и харденинга программа устраняет уязвимости и мисконфигурации, уменьшая поверхность атаки и минимизируя вероятность эксплуатации

типовых техник злоумышленника, тем самым защищая от сразу нескольких тактик злоумышленника по MITRE ATT&CK:

- **TA0001-Initial Access** (T1190-Exploit Public-Facing Application)
- **TA0004-Privilege Escalation** (T1068-Exploitation for Privilege Escalation, T1548.003-Abuse Elevation Control Mechanism: Sudo and Sudo Caching)
- **TA0005-Defense Evasion** (T1548.003-Abuse Elevation Control Mechanism: Sudo and Sudo Caching, T1562.004-Impair Defenses: Disable or Modify System Firewall)

3. ОПИСАНИЕ РЕАЛИЗАЦИИ

3.1 Техническое задание

Идея проекта:

Разрабатываемое программное средство предназначено для автоматизации аудита и харденинга операционной системы с использованием открытых стандартов SCAP. Его применение позволяет упростить и ускорить нахождение уязвимостей и мисконфигураций, обеспечить воспроизводимость результатов проверки и переносимость контента между дистрибутивами и инструментами. В отличие от решений, основанных на проприетарных базах и закрытых форматах, проектная программа опирается на SCAP-контент (OVAL, XCCDF, DataStream) как на стандартизированный способ описания правил аудита и устранения найденных несоответствий.

Цель проекта:

Разработать программное средство для автоматизированной проверки системы на наличие уязвимостей и мисконфигураций на основе стандартизированного SCAP-контента (OVAL/XCCDF/DS), с формированием табличного и обычного отчёта и возможностью автоматического устранения несоответствий (remediation).

Задачи разработки:

1. Обеспечить загрузку и подготовку SCAP-контента для выбранных операционных систем.
2. Реализовать выполнение проверок по OVAL-дефинициям с получением статусов результатов.
3. Реализовать выполнение проверок по XCCDF-контенту (профили/правила) с получением статусов результатов.
4. Реализовать механизм автоматического патчинга для адаптации XCCDF-контента под целевую систему при несовпадении идентификатора системы с указанным в контенте.
5. Реализовать применение remediation для правил XCCDF со статусом fail при наличии соответствующего контента.
6. Реализовать формирование отчёта в табличном виде и отображение результатов пользователю через графический интерфейс.

Требования к функциональности:

1. Загрузка SCAP-контента: Программа должна давать возможность выбора операционной системы и загрузки соответствующего SCAP-контента.
2. Выполнение OVAL-проверок: Программа должна интерпретировать OVAL-дефиниции и выполнять проверки состояния системы.
3. Выполнение XCCDF-проверок: Программа должна интерпретировать XCCDF-правила и выполнять проверки состояния системы.
4. Патчинг: при несовпадении идентификаторов платформы в контенте и на целевой системе программа должна выполнять автоматический патчинг, обеспечивая применимость выбранного XCCDF-контента на целевой ОС.
5. Remediation: для правил со статусом fail программа должна применять remediation-действия при наличии соответствующего remediation-контента.
6. Отчёт и вывод результатов: Программа должна формировать отчёт в табличном виде, включающий: идентификатор проверки (OVAL/XCCDF), описание, статус, уровень важности (если присутствует в XCCDF).

Следующим этапом составления технического задания является выбор используемых технологий. Этот этап можно считать по-настоящему важным, так как от выбранных инструментов зависят многие характеристики конечного продукта. Мы разрабатываем десктопное приложение, вследствие чего выбор фреймворка графического интерфейса является довольно важным этапом. Для удобного просмотра основных параметров фреймворка была сделана таблица.

Таблица 2 - Сравнение фреймворков

Наименование фреймворка, иллюстрация	Поддерживаемые языки	Лицензия	Производительность	размер конечного приложения	Поддерживаемые платформы	Потребление оперативной памяти
 Electron.js	JavaScript	MIT	Низкая	Большой	Windows, macOS, Linux	Высокое
 Flutter	Dart	BSD	Высокая	Средний-большой	Windows, macOS, Linux, Android, iOS, Web	Среднее
 GTK	20+ языков, включая c++ и python	LPGL	Средняя (зависит от ОС)	Низкий-средний	Linux (нативно), Windows и macOS.	Низкое-среднее (зависит от ОС)
 Qt	5 языков, включая c++ и python	LPGL, GPL	Высокая	Средний	Windows, macOS, Linux, Android, iOS	Низкое

Самыми главными параметрами при выборе можно по праву считать поддерживаемые платформы и лицензию. Для начала следует разобрать вопрос лицензий. Все виды лицензий (кроме GPL) в таблице почти не накладывают ограничений на использование в коммерческих продуктах. Также, как можно видеть в таблице, GTK имеет слабую поддержку ОС помимо линукса, а Electron.js имеет высокое потребление ресурсов из-за использования Chromium. В итоге нам стоит рассмотреть только 2 фреймворка – Flutter и Qt. По своим параметрам они очень похожи, но отличия возникают при их ближайшем рассмотрении: Flutter поддерживает только язык программирования Dart, в то время как на Qt можно писать почти на всех наиболее распространённых языках, что уменьшит время разработки. Также Flutter проигрывает Qt в аспектах веса конечной программы и потребления оперативной памяти. После выбора фреймворка было решено писать на Python, так как по сравнению с C++ с ним будет проще и быстрее работать.

После выбора языка перед нами появляется проблема в виде того, что библиотека PyQt для работы с Qt распространяется по лицензии GPL, что не позволит использовать её в коммерческих проектах. Но также существует и библиотека PySide, которая почти полностью идентична PyQt, но при этом распространяется по лицензии LGPL, соответственно использовать мы будем её.

В качестве инструмента для разработки графического интерфейса программы будет использоваться Qt Designer, а для реализации функционала программы будут использоваться следующие модули: subprocess, xml.etree.ElementTree, sys.

Первая версия программы будет разрабатываться под ОС Linux, так как на Windows SCAP не возымел должной популярности, вследствие чего поддержка данного протокола была прекращена как со стороны как Microsoft, так и со стороны разработчиков инструментария SCAP. Например, OpenSCAP прекратил поддержку ОС Windows в 2022 году. Второй же важной причиной является то, что основной аудиторией программы являются Системные Администраторы, а по официальной статистике около 77% серверов работают под управлением ОС Linux.

Далее идёт выбор инструмента для обработки SCAP контента. Этот этап является критичным, так как от выбранного инструментария будет зависеть большинство характеристик программы. Для обоснования выбора приведена Таблица 3. По ней мы можем видеть, что SCC не подходит нам, потому что работает только на Windows, Ovaldi не подходит из-за отсутствия обновлений и поддержки с 2015, вследствие чего он не будет работать без дописывания его функционала вручную. А самодельный SCAP-интерпретатор не имеет смысла писать по многим причинам, начиная от технической сложности интерпретаторов, заканчивая тем, что он будет работать медленней других аналогичных разработок. В итоге для разработки программы в качестве движка для обработки SCAP контента выбран openSCAP.

Таблица 3 - Сравнение SCAP-интерпретаторов

Движок	Скорость проверки	Актуальность функционала	Возможность интеграции в программу	Поддерживаемые ОС
OpenSCAP	Высокая	Высокая	Средняя	Linux
SCC (SCAP Compliance Checker)	Низкая	Высокая	Низкая	Windows
Самодельный	Низкая	Низкая	Высокая	Linux/Windows
Ovaldi	Высокая	Низкая	Средняя	Linux, Windows

Методология тестирования готового продукта: должно быть проведено два этапа тестирования продукта – функциональное тестирование, покрывающее весь функционал программы в разной среде, и пользовательское тестирование, проведённое в организации в реальных условиях.

3.2 Принцип работы программы

После выбора технологического стека и описания технического задания можно описать конечный принцип работы программы по пунктам. Начальным этапом работы является запуск программы, после чего идёт инициализация интерфейса и переход в основную функцию работы программы. Далее пользователь выбирает свою ОС в списке поддерживаемых. После этого, пользователь выбирает интересующий его функционал (XCCDF/OVAL). После нажатия кнопки запуска проверки программа начинает выполнять цепочку действий:

1. Выполняется загрузка SCAP-контента для функционирования программы. Если загрузка не проходит успешно – программа совершает выход из программы с ошибкой. При успешной же загрузке программа переходит к следующему шагу.
2. Далее происходит автоматический патчинг SCAP-контента если CPE системы не соответствует указанному в самом контенте
3. Начинается сама OVAL/XCCDF проверка, при возникновении ошибок они выводятся в консоль.
4. Если при выборе проверки по XCCDF правилам было выбрано проведение исправления найденных уязвимостей/мисконфигураций – то перед формированием отчёта будет проведен remediation по контенту из XCCDF-правил.
5. Перед выводом данных проверки в удобочитаемом виде они сначала агрегируются в списки
6. Вывод данных проверки на экран в табличном, удобочитаемом для чтения и просмотра виде.

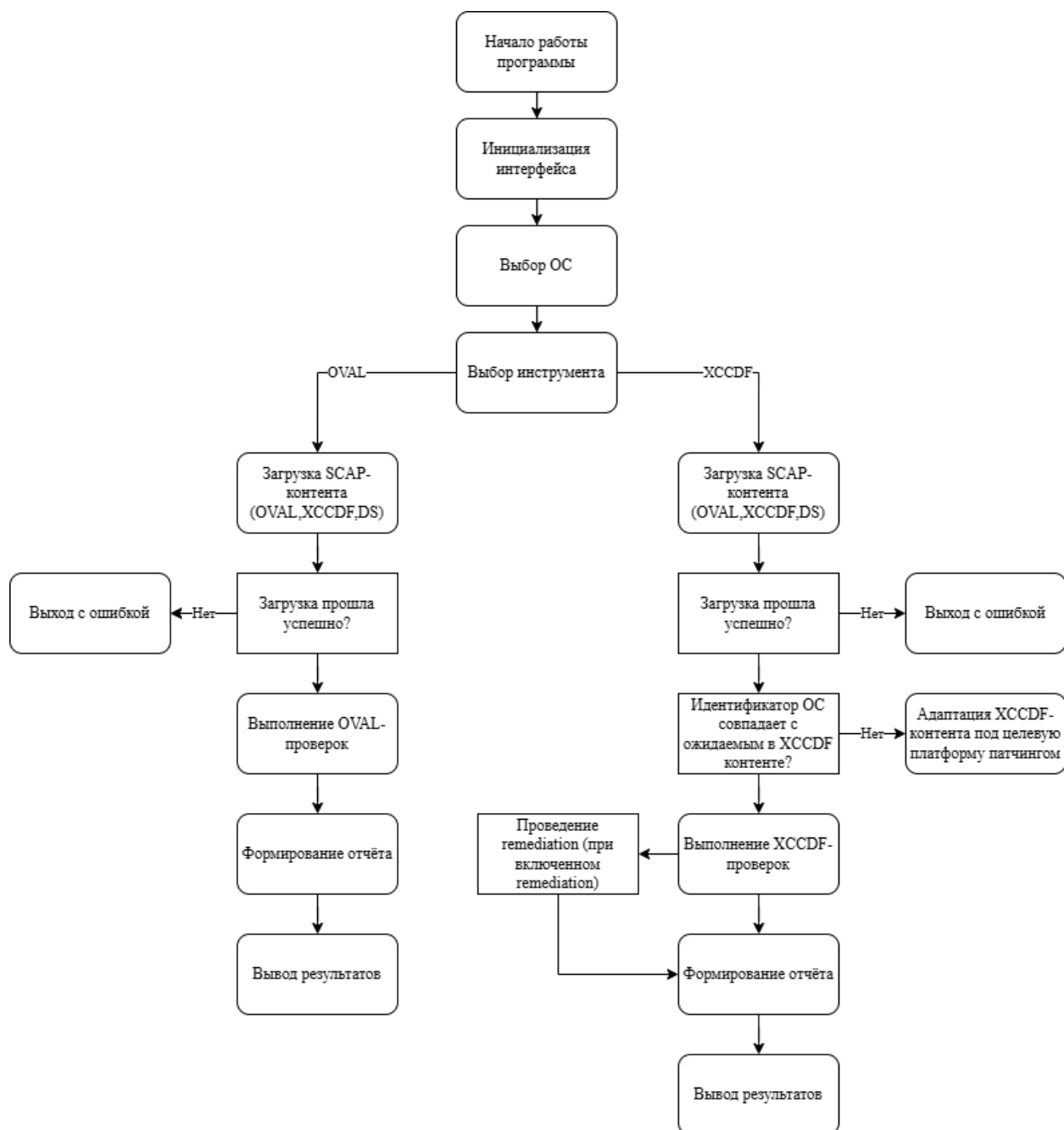


Рисунок 3.1 - Принцип работы программы

3.3 Доказательство работоспособности

Работоспособность разработанной программы подтверждается тем, что она реализует решение задачи в том виде, в каком она была формально поставлена в разделах 2.1–2.2: определены входные данные S, C, P, выходные данные Result_OVAL, Result_XCCDF, Report и допустимые статусы результатов. Программа выполняет полный цикл: загрузка SCAP-контента, патчинг при необходимости, выполнение проверок, применение remediation и формирование итогового отчёта.

Формальная верификация в рамках данной работы заключается в проверке соответствия реализации указанной формальной модели: для каждого элемента входа должен быть получен корректный элемент выхода, а значения результатов должны принадлежать заранее заданным множествам статусов. Это проверяется следующим образом:

1. Проверка корректности результатов OVAL. Для каждой OVAL-дефиниции d_i программа вычисляет $\text{Result_OVAL}(d_i)$ и присваивает ему один из формально заданных статусов $\{\text{true}, \text{false}, \text{unknown}, \text{error}, \text{notapplicable}\}$. Полученные значения сохраняются и отображаются в отчёте, что обеспечивает проверяемость и повторяемость результата.
2. Проверка корректности результатов XCCDF. Для каждого XCCDF-правила r_i в рамках выбранного профиля вычисляется $\text{Result_XCCDF}(r_i)$ из множества $\{\text{pass}, \text{fail}, \text{notselected}, \text{notapplicable}, \text{error}\}$. Полученные значения сохраняются и отображаются в отчёте, что также обеспечивает проверяемость и повторяемость результата.
3. Проверка патчинга. Если исходный контент неприменим к целевой платформе из-за отличающихся идентификаторов, выполняется патчинг, после чего контент становится применимым и корректно проходит этап вычисления результатов.
4. Проверка remediation. Для правил со статусом fail, при наличии remediation-контента, программа выполняет действие исправления.

Таким образом, программа демонстрирует работоспособность не только на уровне запуска и получения отчёта, но и на уровне соответствия формальной постановке задачи: входные данные преобразуются в определённые формально выходные результаты, а remediation приводит систему к состоянию, которое подтверждается повторной проверкой

4. Тестирование

4.1 Цели тестирования

Было проведено два этапа тестирования продукта – функциональное тестирование, покрывающее весь функционал программы в разной среде, и пользовательское тестирование, проведённое в организации в реальных условиях.

Цели функционального тестирования:

1. Проверка готовности программы к использованию
2. Проверка соответствия требованиям функциональности из технического задания
3. Выявление ошибок и недочётов программы.
4. Выявление потенциальных векторов развития
5. Проверка поведения программы в граничных случаях

Цели пользовательского тестирования:

1. Проверка готовности программы к использованию
2. Выявление ошибок и недочётов программы.
3. Получение обратной связи по работе программы
4. Выявление потенциальных векторов развития путём опроса организации, проводившей тестирование программ

4.2 Функциональное тестирование

Функциональное тестирование было основано на наборе тестов, покрывающих основной функционал программы: проверку по OVAL-дефинициям и XCCDF-правилам, автоматический патчинг и remediation. В качестве операционных систем были взяты: Debian 12, Red Hat Linux Enterprise 9.6, Rocky Linux 9.7, а также Kali Linux.

Подробное описание тестов:

1. Тест проверки интерпретации OVAL-дефиниций на Debian 12 был проведён для подтверждения работы базового функционала программы, так как ожидаемый вывод был заранее известен. Единственной отличительной чертой данного теста является довольно большое количество выходных данных для вывода в пользовательский интерфейс.
2. Тест проверки интерпретации XCCDF-правил на Red Hat Linux Enterprise 9.6 был проведён также ради проверки базового функционала продукта. В качестве профиля для работы был выбран `xccdf_org.ssgproject.content_profile_cis`, так как он является

стандартом для проверки систем. Как и было написано в ожидаемом результате – в результатах проверки присутствовали только статусы pass и fail.

3. Этот тест является демонстрацией сценария запуска XCCDF-правил, созданных для другой ОС без патчинга. Данный тест был проведён на Rocky Linux 9.7 с идентичным контентом и профилем из прошлого теста и без патчинга. Так как в разном SCAP-контенте присутствуют свои проверки CPE – нельзя было заранее задать ожидаемый результат, так как проверка могла как не начаться совсем, так и закончиться частично удачно (например, некоторые правила могли бы пройти проверку и выдать fail и pass). В данном случае проверка была успешно пройдена, но все правила вернули статус notapplicable.
4. Данный тест идентичен прошлому, за исключением того, что теперь к SCAP-контенту применяется патчинг. Патчинг в теории может привести как к полной успешной проверке, так и к частичной успешной проверке. В качестве ожидаемого результата был выбран самый оптимистичный вариант – все статусы проверок вернут pass или fail, но как мы можем видеть по полученному результату – 150 проверок вернули pass, 196 вернули fail, и 29 вернули notapplicable.
5. Последний тест проверяет совмещение автоматического патчинга и включённого исправления найденных уязвимостей/мисконфигураций. В качестве ожидаемого результата был взят неизвестный результат, так как из-за применяемых исправлений программа может как вернуть ошибку, так и успешно завершить remediation. Как можно видеть по результату теста – из 44 статусов проверок 22 вернули pass, 17 fail и 5 notapplicable, после чего из 17 несоответствий было успешно исправлено 13. 4 оставшихся несоответствия не были исправлены лишь из-за того, что в SCAP-контенте не было исправлений для них. Также, во время проведения этого теста было замечено, что если remediation не находит правил для исправления (например, если все возможные правила уже были исправлены на первом проходе) – то он возвращает ошибку несовпадения платформы.

Таблица 4 - Функциональное тестирование

Суть теста	Ожидаемый результат	Получаемый результат
Выполнение проверки OVAL-дефиниций на Debian 12	Все статусы проверок дефиниций вернут true или false	Все статусы проверок дефиниций содержали true или false. Проверка проверила 1428 дефиниц, 1279 из которых вернули false и 149 вернули true. (одна дефиниция проверяла 1 или сразу несколько CVE).
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_cis на Red Hat Linux Enterprise 9.6	Все статусы проверок XCCDF-правил вернут pass или fail	Все статусы проверок XCCDF-правил вернули pass или fail. Всего прошло тестирование 375 правил, 170 из которых вернули pass и 205 вернули fail.
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_cis на Rocky Linux 9.7 без автоматического патчинга	Неизвестный результат	Все статусы проверок XCCDF-правил вернули not applicable, хоть Rocky Linux и основан на RHEL. Произошло это из-за несовпадения идентификатора ОС.
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_cis на Rocky Linux 9.7 с автоматическим патчингом	Все статусы проверок XCCDF-правил вернут pass или fail	Из 375 статусов проверок XCCDF-правил 29 вернули not applicable, 150 вернули pass, 196 вернули fail.
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_standard на Kali Linux с автоматическим патчингом и последующим remediation	Неизвестный результат	Из 44 статусов проверок 22 вернули pass, 17 fail и 5 not applicable. После проверки 13 несоответствий были успешно исправлены, успех исправления составил 100% относительно имеющегося контента для исправления.

4.3 Пользовательское тестирование

Пользовательское тестирование было проведено на базе Муниципального бюджетного образовательного учреждения Гимназия №1. Тестирование было проведено на 10 персональных компьютерах, на каждом из которых была установлена виртуальная машина с Kali Linux. На каждом из компьютеров тестирование было проведено с одинаковыми настройками программы: режим проверки по XCCDF-правилам, выбранный профиль – `xccdf_org.ssgproject.content_profile_standard`, включённый `remediation`. По результатам тестирования на всех 10 системах результаты проверки и исправления найденных несоответствий были идентичны – из 44 правил изначально 25 вернули `pass`, 14 вернули `fail` и 5 `notapplicable`. После проведённого `remediation` из 44 правил 35 начали возвращать `pass`, 4 `fail` и 5 `notapplicable`.

4.4 Анализ результатов тестирования

Анализ результатов тестирования стоит провести относительно целей функционального и пользовательского тестирования. Функциональное тестирование:

1. Основываясь на результатах тестов – все основные функции программы работают интуитивно понятно и безошибочно, вследствие чего программа готова к использованию.
2. Программа успешно совершает загрузку SCAP-контента, проверки по OVAL-дефинициям и XCCDF-правилам, автоматический патчинг и проведение `remediation`, а также исправно выводит результат проверки в графический интерфейс, вследствие чего все требования к функционалу программы из технического задания можно считать выполненными.
3. Во время тестов был найден недочёт проведения исправления несоответствий после патчинга SCAP-контента. Как показано в последнем тесте из функционального тестирования – если после исправления найденных несоответствий запустить программу по второму циклу с теми же настройками – то после этапа `remediation` программа выдаст ошибку несовпадения платформы, которая по идее должна исправляться автоматическим патчингом. Возникновение этой ошибки никак не влияет на получение результатов проверки, но добавляет дополнительное условие в реализации программы – при ошибке во время проверки программа не должна завершать работу. (соответственно все ошибки во время проверки должны быть выписаны в логи/выведены в виде информационного окна с ошибкой)
4. По результатам функционального тестирования было выведено два потенциальных вектора развития программы: расширение доступных дистрибутивов/источников и

улучшение функционала программы в виде добавления расширенной фильтрации и визуального отображения проведённого remediation.

5. Среди тестов был рассмотрен единственный граничный случай для проектной программы, а именно совмещение автоматического патчинга и remediation. Проверка данного граничного случая позволила найти недочёт в совмещении патчинга и исправления несоответствий, подробное описание данного недочёта было представлено ранее.

Пользовательское тестирование:

1. Аналогично функциональному тестированию все основные функции программы работают интуитивно понятно и безошибочно, вследствие чего программа готова к использованию.
2. Ошибок и недочётов во время пользовательского тестирования выявлено не было.
3. Тестирующая организация положительно оценила работу программы и пользу от неё.
4. Организация выразила пожелание дальнейшего развития системы работы с отчётами в рамках программы, в особенности добавление системы расширенной фильтрации результатов.

4.5 Перспективы развития проекта

Разработанная программа ввиду своей модульной архитектуры как на стороне визуального интерфейса, так и на стороне функционала - позволяет быстро изменять и расширять функционал продукта. Из основных направлений дальнейшего развития проекта можно выделить:

1. Расширение поддерживаемых дистрибутивов и источников SCAP-контента. Возможна поддержка большего числа дистрибутивов, а также возможно увеличение количества источников SCAP-контента.
2. Для удобства просмотра и анализа отчётов внутри программы может быть реализована расширенная система фильтрации результатов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была достигнута поставленная цель - создано программное средство автоматизированного аудита и харденинга операционной системы на основе SCAP-контента (OVAL/XCCDF).

Для достижения цели были решены следующие задачи:

1. Реализован визуальный интерфейс программы, обеспечивающий запуск проверок по SCAP-контенту и просмотр результатов в удобочитаемом виде.
2. Разработан модуль выполнения проверок по OVAL-дефинициям, обеспечивающий интерпретацию условий и получение статусов результатов.
3. Разработан модуль выполнения проверок по XCCDF-контенту, позволяющий проводить оценку правил по выбранному профилю и формировать результаты проверки.
4. Реализован механизм автоматического патчинга, обеспечивающий адаптацию XCCDF-контента к целевой системе при несовпадении идентификаторов платформы.
5. Добавлена поддержка remediation для XCCDF, позволяющая применять корректирующие действия при наличии соответствующего контента.
6. Реализован модуль подгрузки SCAP-контента для выбранной операционной системы и подготовки контента к выполнению проверок.
7. Реализована агрегация результатов проверки из XML-файлов с последующим выводом в табличном виде.

Проект является перспективным для практического применения, поскольку опирается на открытые стандарты SCAP, обеспечивает воспроизводимость проверок и снижает зависимость от проприетарных решений. Модульная архитектура программы позволяет расширять функциональность: добавлять поддержку новых платформ и источников SCAP-контента, улучшать отчётность, а также развивать механизмы автоматизации харденинга. Полная адаптируемость продукта достигнута за счёт нескольких факторов: стандартизированного результата проверок в SCAP-формате и возможности экспорта полученных таблиц в табличный формат. Эти две возможности полностью покрывают возможные направления интеграции – в автоматизированный процесс (результаты в SCAP-формате обрабатываются сторонними инструментами) и ручной (анализ таблиц

аудиторами безопасности). Проактивность продукта достигается за счёт автоматического обновления баз данных SCAP-контента.

Вместе с тем текущая версия имеет ряд ограничений. В частности, после применения remediation в визуальном интерфейсе не выделяются исправленные несоответствия, что снижает удобство подтверждения устранения несоответствий и может быть улучшено в дальнейшем. Также требуют развития возможности отчётности (расширенная фильтрация в рамках программы) и расширение набора поддерживаемых дистрибутивов/контента.

Таким образом, разработанное решение демонстрирует работоспособность и практическую применимость для задач регулярного аудита и харденинга операционных систем, а обозначенные направления развития позволяют повысить уровень автоматизации и удобство эксплуатации в дальнейшем.

СПИСОК ЛИТЕРАТУРЫ

1. OpenSCAP User Manual [Электронный ресурс] // static.open-scap.org. — 2025. — URL: https://static.open-scap.org/openscap-1.3/oscap_user_manual.html (дата обращения 02.01.2026).
2. Аудит информационной безопасности. SCAP ликбез [Электронный ресурс] // habr.com. — 2021. — URL: <https://habr.com/ru/articles/537974/> (дата обращения 02.01.2026).
3. Введение в OVAL: Open vulnerability and Assessment Language [Электронный ресурс] // habr.com. — 2012. — URL: <https://habr.com/ru/articles/136046/> (дата обращения 02.01.2026).
4. Securing the open source supply chain: The essential role of CVEs [Электронный ресурс] // github.blog. — 2024. — URL: <https://github.blog/security/supply-chain-security/securing-the-open-source-supply-chain-the-essential-role-of-cves/> (дата обращения 03.01.2026).
5. Usage share of operating systems [Электронный ресурс] // en.wikipedia.org. — 2024. — URL: https://en.wikipedia.org/wiki/Usage_share_of_operating_systems (дата обращения 03.01.2026).
6. Российские сканеры уязвимостей: обзор технологий и решений [Электронный ресурс] // www.securitylab.ru. — 2024. — URL: <https://www.securitylab.ru/blog/personal/paragraph/354220.php> (дата обращения 03.01.2026).
7. Obligations of the GPL and LGPL [Электронный ресурс] // qt.io. — 2025. — URL: <https://www.qt.io/licensing/open-source-lgpl-obligations> (дата обращения 03.01.2026).
8. Аудит информационной безопасности. XCCDF и OVAL [Электронный ресурс] // habr.com. — 2021. — URL: <https://habr.com/ru/articles/538764/> (дата обращения 04.01.2026).
9. Security Content Automation Protocol [Электронный ресурс] // csrc.nist.gov. — 2026. — URL: <https://csrc.nist.gov/projects/security-content-automation-protocol> (дата обращения 04.01.2026).
10. MITRE ATT&CK. Globally-accessible knowledge base of adversary tactics and techniques based on real-world observations. [Электронный ресурс] // attack.mitre.org. — 2025. — URL: <https://attack.mitre.org/> (дата обращения 05.01.2026).
11. Scanning the system for security compliance and vulnerabilities [Электронный ресурс] // docs.redhat.com. — 2024. — URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/system_design_g

uide/scanning_the_system_for_security_compliance_and_vulnerabilities#configuration-compliance-tools-in-rhel_system-design-guide (дата обращения 05.01.2026).

12. Using OpenSCAP to Remediate the System [Электронный ресурс] // docs.redhat.com. — 2018. — URL: https://docs.redhat.com/de/documentation/red_hat_enterprise_linux/6/html/security_guide/sect-using_openscap_to_remediate_the_system (дата обращения 05.01.2026).

ПРИЛОЖЕНИЕ А

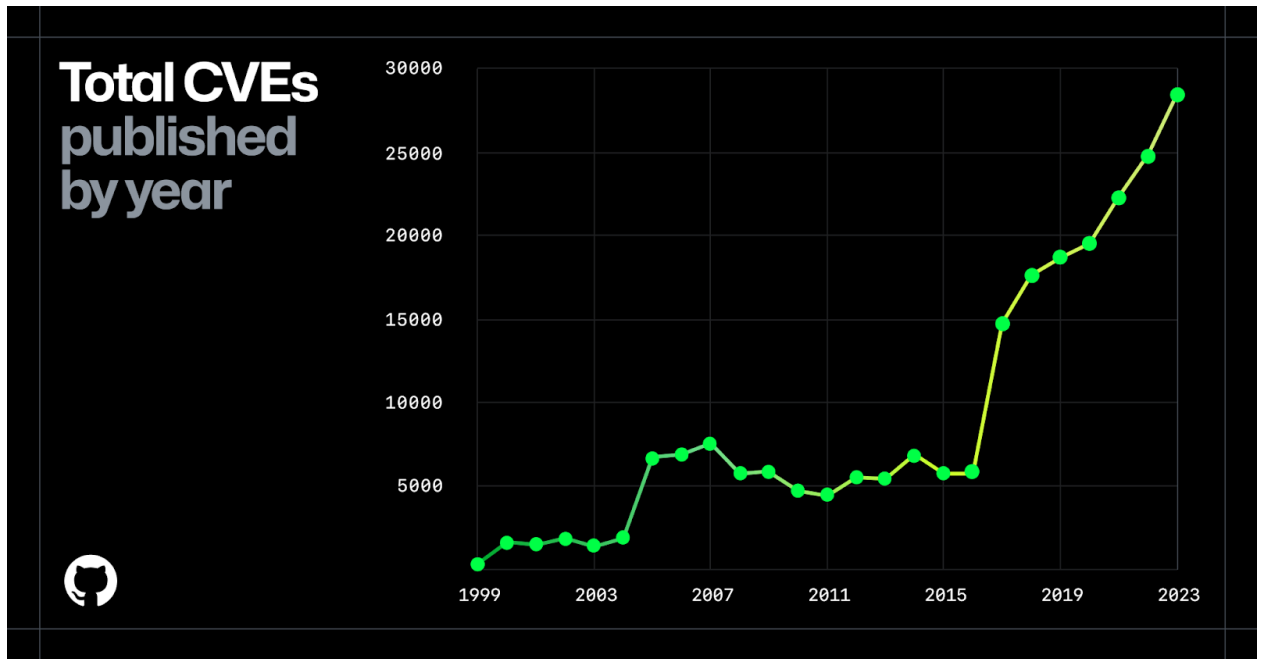


Рисунок 1.1 - График количества найденных уязвимостей по годам

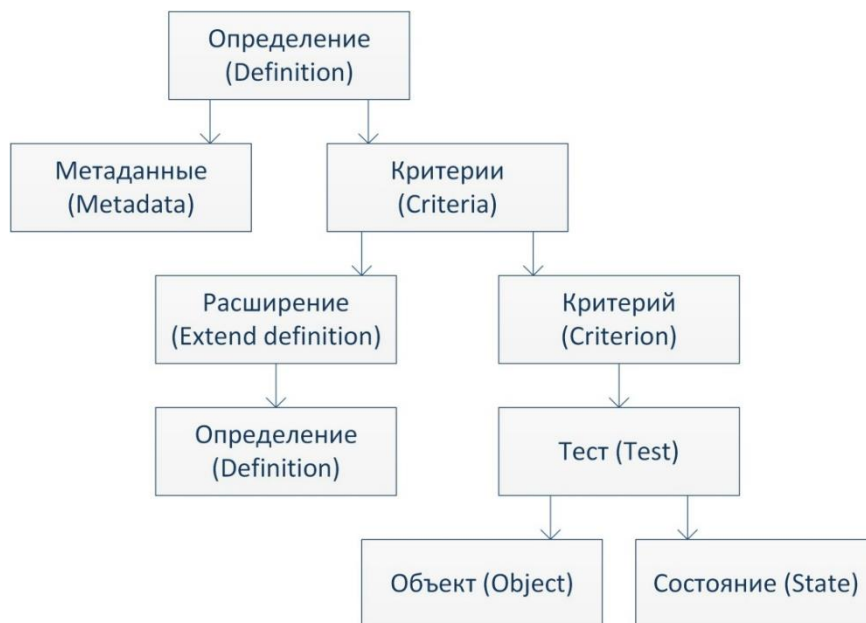


Рисунок 1.2 - Структура OVAL



Рисунок 1.3 - Структура XCCDF



Рисунок 1.4 - Процесс работы протокола SCAP

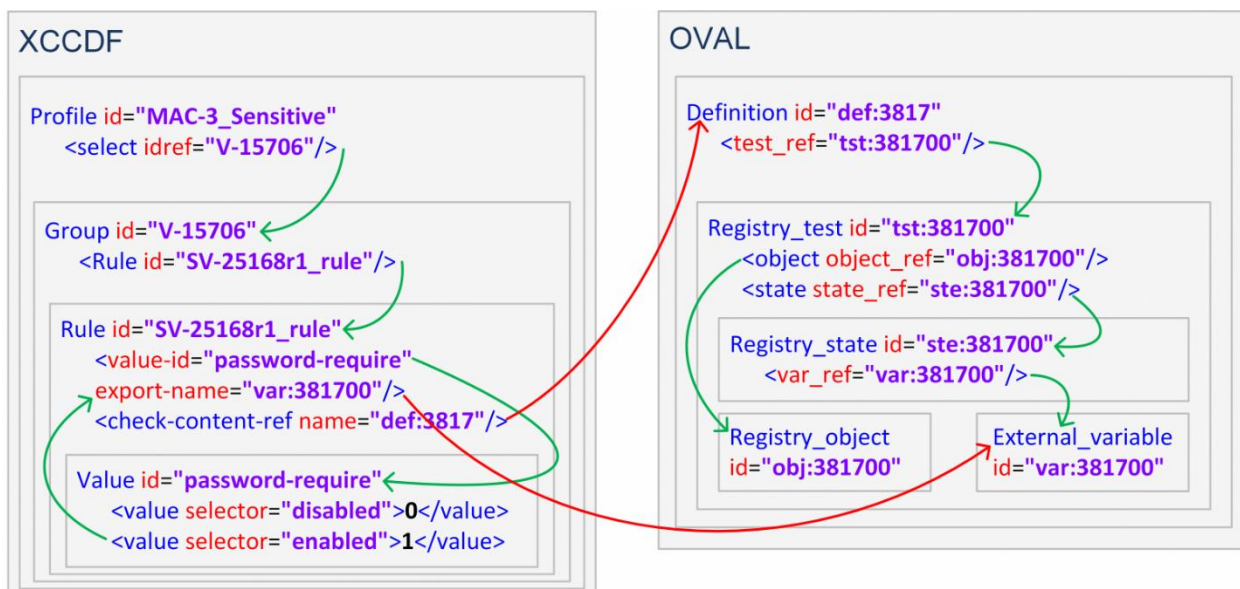


Рисунок 1.5 - Структура взаимодействия XCCDF и OVAL на примере CIS Benchmark

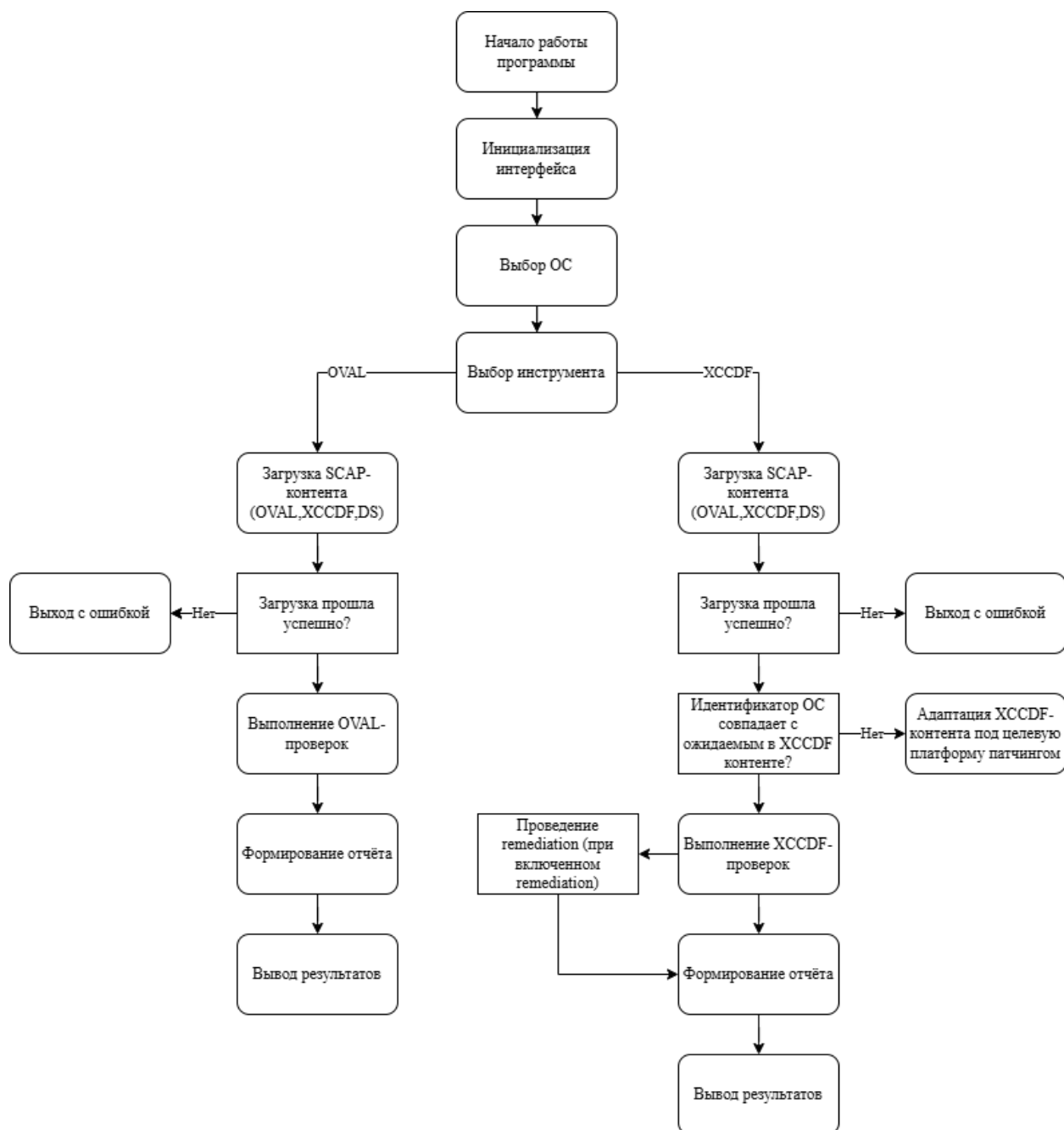


Рисунок 3.1 - Принцип работы программы

Таблица 1 - Сравнение с аналогами

Инструмент	Поддержка SCAP	OpenSource	Цена	Remediation	Tailoring
Xspider	Нет	Нет	Платно	Нет	Нет
RedCheck	Да	Нет	Платно	Да	Нет
ScanOVAL	Частичная (только OVAL-контент от ФСТЭК)	Нет	Бесплатно	Нет	Нет
MaxPatrol VM	Нет	Нет	Платно	Да	Нет
Проектная программа	Да	Да	Бесплатно	Да	Да

Таблица 2 - Сравнение фреймворков

Наименование фреймворка, иллюстрация	Поддерживаемые языки	Лицензия	Производительность	размер конечного приложения	Поддерживаемые платформы	Потребление оперативной памяти
 Electron js	JavaScript	MIT	Низкая	Большой	Windows, macOS, Linux	Высокое
 Flutter	Dart	BSD	Высокая	Средний-большой	Windows, macOS, Linux, Android, iOS, Web	Среднее
 GTK	20+ языков, включая с++ и python	LPGL	Средняя (зависит от ОС)	Низкий-средний	Linux (нативно), Windows и macOS.	Низкое-среднее (зависит от ОС)
 Qt	5 языков, включая с++ и python	LPGL, GPL	Высокая	Средний	Windows, macOS, Linux, Android, iOS	Низкое

Таблица 3 - Сравнение SCAP-интерпретаторов

Движок	Скорость проверки	Актуальность функционала	Возможность интеграции в программу	Поддерживаемые ОС
OpenSCAP	Высокая	Высокая	Средняя	Linux
SCC (SCAP Compliance Checker)	Низкая	Высокая	Низкая	Windows
Самодельный	Низкая	Низкая	Высокая	Linux/Windows
Ovaldi	Высокая	Низкая	Средняя	Linux, Windows

Таблица 4 - Функциональное тестирование

Суть теста	Ожидаемый результат	Получаемый результат
Выполнение проверки OVAL-дефиниций на Debian 12	Все статусы проверок дефиниций вернут true или false	Все статусы проверок дефиниций содержали true или false. Проверка проверила 1428 дефиниц, 1279 из которых вернули false и 149 вернули true. (одна дефиниция проверяла 1 или сразу несколько CVE).
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_cis на Red Hat Linux Enterprise 9.6	Все статусы проверок XCCDF-правил вернут pass или fail	Все статусы проверок XCCDF-правил вернули pass или fail. Всего прошло тестирование 375 правил, 170 из которых вернули pass и 205 вернули fail.
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_cis на Rocky Linux 9.7 без автоматического патчинга	Неизвестный результат	Все статусы проверок XCCDF-правил вернули not applicable, хоть Rocky Linux и основан на RHEL. Произошло это из-за несовпадения идентификатора ОС.
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_cis на Rocky Linux 9.7 с автоматическим патчингом	Все статусы проверок XCCDF-правил вернут pass или fail	Из 375 статусов проверок XCCDF-правил 29 вернули not applicable, 150 вернули pass, 196 вернули fail.
Выполнение проверки XCCDF-правил с профилем xccdf_org.ssgproject.content_profile_standard на Kali Linux с автоматическим патчингом и последующим remediation	Неизвестный результат	Из 44 статусов проверок 22 вернули pass, 17 fail и 5 not applicable. После проверки 13 несоответствий были успешно исправлены, успех исправления составил 100% относительно имеющегося контента для исправления.

ПРИЛОЖЕНИЕ Б

```
class MainWindow(qtw.QWidget, Ui_Form):
    def __init__(self):
        super().__init__()
        self.setupUi(self)
        self.bt_OVALcheck.clicked.connect(self.OVAL_check)
        self.bt_XCCDFcheck.clicked.connect(self.XCCDF_check)
        self.bt_OVAL_page.clicked.connect(lambda:
self.stackedWidget.setCurrentWidget(self.OVAL_page))
        self.bt_XCCDF_page.clicked.connect(lambda:
self.stackedWidget.setCurrentWidget(self.XCCDF_page))
        self.first_encounter = 0

    def OVAL_check(self):
        if self.first_encounter == 0:
            self.first_encounter = 1
            download_SCAP_content(self.Cb_chooseos.currentText())
            patch_SCAP_content(self.Cb_chooseos.currentText())
            do_oval_scan(self.Cb_chooseos.currentText())
            file_name = "results_oval.xml"
            results = parse_oval(file_name)
            for v in results:
                row_position = self.table_OVAL_results.rowCount()
                self.table_OVAL_results.insertRow(row_position)
                self.table_OVAL_results.setItem(row_position, 0,
qtw.QTableWidgetItem(v['cve']))
                self.table_OVAL_results.setItem(row_position, 1,
qtw.QTableWidgetItem(v['result']))
                self.table_OVAL_results.setItem(row_position, 2,
qtw.QTableWidgetItem(v['description']))

    def XCCDF_check(self):
        if self.first_encounter == 0:
            self.first_encounter = 1
            download_SCAP_content(self.Cb_chooseos_2.currentText())
            patch_SCAP_content(self.Cb_chooseos_2.currentText())
            do_xccdf_scan(self.Cb_chooseos_2.currentText(),
self.chk_remediation.isChecked())
            file_name = "results_xccdf.xml"
            results = parse_xccdf(file_name)
            for v in results:
                row_position = self.table_XCCDF_results.rowCount()
                self.table_XCCDF_results.insertRow(row_position)
                self.table_XCCDF_results.setItem(row_position, 0,
qtw.QTableWidgetItem(v['title']))
                self.table_XCCDF_results.setItem(row_position, 1,
qtw.QTableWidgetItem(v['result']))
                self.table_XCCDF_results.setItem(row_position, 2,
qtw.QTableWidgetItem(v['severity']))
                self.table_XCCDF_results.setItem(row_position, 3,
qtw.QTableWidgetItem(v['description']))
```

Основной цикл работы программы

```
def apt_install(package_name: str):
    try:
        subprocess.run(["apt", "reinstall", "-y",
package_name], check=True, capture_output=True, text=True)
    except subprocess.CalledProcessError as error:
        stderr = error.stderr
        raise SystemExit(f"{package_name} {stderr}")

def download_SCAP_content(system_cpe: str):
    if system_cpe == "Kali Linux":
        apt_install("openscap-scanner")
        apt_install("ssg-debian")
```

Функция скачивания SCAP-контента (на примере Kali Linux)

```
def patch_SCAP_content(system_cpe: str):
    if system_cpe == "Kali Linux":
        path = Path(r"/usr/share/xml/scap/ssg/content/ssg-debian13-ds.xml")
        text = path.read_text(encoding="utf-8")
        pattern = re.compile(r'^\s*<oval-def:criterion comment="Debian13 is
installed" test_ref="oval:ssg-test_debian_13:tst:1"/>\r?\n?', re.MULTILINE)
        # регулярка сверху пропускает сколько угодно пробелов до самого текста,
        a re.MULTILINE применяет такую операцию для каждой строки
        new_text, count = pattern.subn("", text, count=1) # выполняет
замену на пустую строку
        if count != 0:
            print(f"при патчинге было изменено {count} строк")
        if count == 0:
            print("патчинг не прошёл")
        backup = path.with_suffix(path.suffix + ".bak")
        backup.write_text(text, encoding="utf-8")
        path.write_text(new_text, encoding="utf-8")
```

Функция автоматического патчинга (на примере Kali Linux)

```
def do_xccdf_scan(system_cpe: str, remediation: int):
    if system_cpe == "Kali Linux":
        try:
            if remediation == 1:
                subprocess.run(
                    [
                        "oscap", "xccdf", "eval",
                        "--skip-signature-validation",
                        "--profile",
                        "xccdf_org.ssgproject.content_profile_standard",
                        "--results", "results_xccdf.xml",
                        "--report", "report_xccdf.html",
                        "--remediate",
                        "/usr/share/xml/scap/ssg/content/ssg-debian13-
ds.xml",
                    ],
                    capture_output=True,
                    text=True,
                )
            else:
                subprocess.run(
                    [
                        "oscap", "xccdf", "eval",
```

```

        "--skip-signature-validation",
        "--profile",
        "xccdf_org.ssgproject.content_profile_standard",
        "--results", "results_xccdf.xml",
        "--report", "report_xccdf.html",
        "/usr/share/xml/scap/ssg/content/ssg-debian13-
ds.xml",
    ],
    capture_output=True,
    text=True,
)

```

Функция проведения проверки по XCCDF-правилам (на примере Kali Linux)

```

def do_oval_scan(system_cpe: str):
    if system_cpe == "Kali Linux":
        try:
            subprocess.run(
                [
                    "oscap", "oval", "eval",
                    "--results", "results_oval.xml",
                    "--report", "report_oval.html",
                    "/usr/share/xml/scap/ssg/content/ssg-debian13-cpe-
oval.xml",
                ],
                capture_output=True,
                text=True,
            )

```

Функция проведения проверки по OVAL-дефинициям (на примере Kali Linux)

```

def parse_oval(file_path):
    try:
        tree = ET.parse(file_path)
        root = tree.getroot() #начальный парсинг XML
    except Exception as e:
        print(e)
        return []

    definitions = {} #тут по айди будут храниться названия и описания
    scan_results = [] #тут по айди будут результаты проверки

    for el in root.findall("./{*}definition"):
        id_of_definition = el.get('id')

        metadata = el.find("./{*}metadata")
        if metadata is not None:
            title = "Unknown"
            description = ""

            title_el = metadata.find("./{*}title")
            if title_el is not None and title_el.text:
                title = title_el.text

            desc_el = metadata.find("./{*}description")
            if desc_el is not None and desc_el.text:
                description = desc_el.text

```

```

        definitions[id_of_definition] = {
            'title': title,
            'description': description
        }
        continue

    res_attr = el.get('result') #если в элементе не находится метадата -
    #значит это результат проверки
    def_id_ref = el.get('definition_id')

    if res_attr and def_id_ref:
        scan_results.append({
            'id': def_id_ref,
            'result': res_attr
        })

    final_out = []

    if not definitions:
        print("в результатах нет definitions")

    for item in scan_results:
        did = item['id']
        meta = definitions.get(did, {}) #пробуем взять по id definition
        #информацию, если не получается - пустой словарь

        final_out.append({
            'id': did,
            'cve': meta.get('title', 'N/A'),
            'result': item['result'],
            'description': meta.get('description', '').replace('\n', ' ')
        })
        #заменяем перенос строк на пробелы и убираем их по краям

    return final_out

```

Функция парсинга результатов из файла результата проверки по OVAL-дефинициям

```

from pathlib import Path
import shlex
import paramiko

remote_xml = "results_xccdf.xml"
remote_datastream = "/usr/share/xml/scap/ssg/content/ssg-debian13-ds.xml"
remote_profile = "xccdf_org.ssgproject.content_profile_standard"
def run_ssh(ssh, password, command, use_sudo=False):
    if use_sudo:
        command = f"echo {shlex.quote(password)} | sudo -S -p '' bash -lc {shlex.quote(command)}"

    _, stdout, stderr = ssh.exec_command(command, get_pty=True)
    out = stdout.read().decode("utf-8", errors="replace")

    return out
def virtual_check(login, password, ip, port):
    port = int(port)

    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

```

```

ssh.connect(
    hostname=ip,
    port=port,
    username=login,
    password=password,
    look_for_keys=False,
    allow_agent=False,
)

try:
    run_ssh(
        ssh,
        password,
        "apt-get update && apt-get install -y --reinstall openscap-
scanner ssg-debian",
        use_sudo=True,
    )

    run_ssh(
        ssh,
        password,
        """python3 -c "from pathlib import Path; import re;
p=Path('/usr/share/xml/scap/ssg/content/ssg-debian13-ds.xml');
t=p.read_text(encoding='utf-8'); n=re.sub(r'^\\s*<oval-def:criterion
comment=\\\"Debian13 is installed\\\" test_ref=\\\"oval:ssg-
test_debian_13:tst:1\\\"/>\\r?\\n?', '', t, count=1, flags=re.MULTILINE);
p.with_suffix(p.suffix + '.bak').write_text(t, encoding='utf-8');
p.write_text(n, encoding='utf-8')\" """,
        use_sudo=True,
    )

    remote_dir = run_ssh(ssh, password, "mktemp -d /tmp/vulnoval-ssh-
XXXXXX").strip()

    run_ssh(
        ssh,
        password,
        " ".join(
            [
                "oscap xccdf eval",
                "--skip-signature-validation",
                f"--profile {remote_profile}",
                f"--results {remote_dir}/{remote_xml}",
                remote_datastream,
            ]
        ),
        use_sudo=True,
    )

    local_xml = Path(remote_xml)
    with ssh.open_sftp() as sftp:
        sftp.get(f"{remote_dir}/{remote_xml}", str(local_xml))

finally:
    ssh.close()

return str(local_xml)

```

Функция проведения проверки по XCCDF-контенту по ssh (на примере Kali Linux)